

Guia de Estudo: Compilador Didático em Python

Fundamentos, estrutura mental, exemplos pequenos e roteiro prático para você desenvolver seu próprio compilador.

Material de apoio ao projeto de Compiladores

Foco: estudar, entender e implementar com autonomia

Sumário

Guia de Estudo: Como Construir um Compilador Didático em Python	3
Como usar este guia	4
Visão geral do compilador	4
A linguagem que você vai compilar	5
Gramática: o contrato da linguagem	5
Fase 1: análise léxica	6
Testes do lexer	8
Fase 2: análise sintática	9
Parser descendente recursivo	9
AST: a árvore do programa	10
Fase 3: análise semântica	11
Tabela de símbolos	12
Verificação de tipos	13
Fase 4: código intermediário	13
Fase 5: código final com bytecode	15
Como ligar as fases	16
Estrutura de pastas recomendada	17
Ordem prática de desenvolvimento	18
Erros que seu compilador deve reportar bem	18
Mini roteiro mental por fase	19
Exemplo completo em fases	20
Otimizações simples	21
O que entregar e como demonstrar	22
Checklist de independência	22
Plano de estudo em 7 dias	23
Conclusão	23

Guia de Estudo: Como Construir um Compilador Didático em Python

Subtítulo: fundamentos, estrutura mental, exemplos pequenos e roteiro prático para você desenvolver seu próprio compilador.

Base do projeto: compilador completo com análise léxica, análise sintática, análise semântica, código intermediário e código final.

Como usar este guia

Este material não é uma solução pronta. A ideia é que você entenda a arquitetura e consiga programar o compilador por conta própria, sabendo o que cada fase recebe, o que ela produz e quais erros deve detectar.

O projeto do enunciado pede um compilador funcional. Para fins didáticos, a escolha mais equilibrada é:

- implementar o compilador em Python;
- criar uma linguagem pequena própria;
- fazer parser descendente recursivo;
- gerar uma representação intermediária;
- gerar bytecode próprio;
- executar o bytecode em uma máquina virtual simples.

Essa escolha evita a complexidade de Assembly real e mantém todos os conceitos importantes de compiladores.

Sacada: um compilador grande parece assustador porque tem muitas fases. Mas cada fase é um tradutor pequeno: texto vira tokens, tokens viram árvore, árvore vira validação, árvore validada vira instruções.

Visão geral do compilador

Um compilador transforma um programa escrito em uma linguagem fonte para uma forma executável ou mais próxima da máquina.

No seu projeto, a cadeia principal pode ser:

```
código fonte
↓
lexer / scanner
↓
tokens
↓
parser
↓
AST
↓
análise semântica
↓
código intermediário
↓
bytecode
↓
máquina virtual
↓
resultado
```

Cada etapa deve ter uma responsabilidade clara:

- lexer: reconhecer símbolos;
- parser: reconhecer estrutura;
- AST: representar o programa;

- semântica: verificar sentido;
- IR: simplificar o programa para geração;
- codegen: transformar IR ou AST em código final;
- VM: executar o código final, se você escolher bytecode.

Sacada: não misture fases. Se o lexer começa a tentar entender `if` inteiro, ele está fazendo trabalho do parser. Se o parser começa a verificar tipo de variável, ele está fazendo trabalho da semântica.

A linguagem que você vai compilar

Você precisa escolher uma linguagem pequena. Ela não precisa ser bonita, precisa ser clara e fácil de implementar.

Um exemplo de programa:

```
int x;
bool grande;

x = read();
grande = x > 10;

if (grande) {
    print(x);
} else {
    print(0);
}

while (x > 0) {
    x = x - 1;
}
```

Recursos mínimos:

- tipos: `int` e `bool`;
- literais: números, `true`, `false`;
- variáveis: declaração e uso;
- comandos: atribuição, `if`, `else`, `while`, `print`;
- entrada: `read()`;
- operações aritméticas: `+`, `-`, `*`, `/`;
- comparações: `==`, `!=`, `<`, `>`;
- blocos: `{ ... }`;
- comentários: `// texto`.

O objetivo não é criar uma linguagem perfeita. O objetivo é criar uma linguagem suficiente para demonstrar todas as fases.

Gramática: o contrato da linguagem

A gramática descreve quais programas são válidos. Ela é a ponte entre a linguagem que você imagina e o parser que você vai escrever.

Uma gramática simples:

```

program      -> declaration* statement* EOF

declaration  -> type IDENT ";"
type         -> "int" | "bool"

statement    -> assignment
              | ifStmt
              | whileStmt
              | printStmt
              | block

assignment   -> IDENT "=" expression ";"
ifStmt       -> "if" "(" expression ")" statement "else" statement
whileStmt    -> "while" "(" expression ")" statement
printStmt    -> "print" "(" expression ")" ";"
block        -> "{" statement* "}"

expression   -> equality
equality     -> comparison (("==" | "!=") comparison)*
comparison   -> term ("<" | ">") term)*
term         -> factor ("+" | "-") factor)*
factor       -> primary ("*" | "/" primary)*
primary      -> NUMBER
              | "true"
              | "false"
              | IDENT
              | "read" "(" ")"
              | "(" expression ")"

```

Essa gramática já resolve precedência de operadores:

- `*` e `/` têm prioridade maior que `+` e `-`;
- comparações vêm depois de expressões aritméticas;
- igualdade vem depois de comparações.

Exemplo:

```
2 + 3 * 4
```

Deve ser entendido como:

```
2 + (3 * 4)
```

e não como:

```
(2 + 3) * 4
```

Sacada: a forma da gramática define a precedência. Você não precisa de uma tabela mágica se organizar as funções do parser por níveis: equality, comparison, term, factor, primary.

Fase 1: análise léxica

A análise léxica, também chamada de scanner ou lexer, transforma caracteres em tokens.

Entrada:

```
int x = 10;
```

Saída conceitual:

```
INT  
IDENT("x")  
EQUAL  
NUMBER(10)  
SEMICOLON  
EOF
```

Um token normalmente tem:

- tipo;
- lexema, que é o texto original;
- valor literal, quando existe;
- linha e coluna, para mensagens de erro.

Exemplo de estrutura em Python:

```
from dataclasses import dataclass  
  
@dataclass  
class Token:  
    type: str  
    lexeme: str  
    literal: object  
    line: int  
    column: int
```

Tipos de token que você provavelmente terá:

```
INT_TYPE, BOOL_TYPE  
IF, ELSE, WHILE, PRINT, READ  
TRUE, FALSE  
IDENT, NUMBER  
PLUS, MINUS, STAR, SLASH  
EQUAL, EQUAL_EQUAL, BANG_EQUAL  
LESS, GREATER  
SEMICOLON, LEFT_PAREN, RIGHT_PAREN  
LEFT_BRACE, RIGHT_BRACE  
EOF
```

O lexer deve ignorar:

- espaços;
- tabs;
- quebras de linha, exceto para contar linha;
- comentários iniciados por `//`.

Um esqueleto mental do scanner, seguindo a organização com `tokens_def.py`:

```
while not self.is_at_end():
    self.start = self.current
    self.scan_token()

self.add_token(tokens_def.EOF)
```

Dentro de `scan_token()`, a primeira ideia é ler um caractere e verificar se ele está na tabela de símbolos simples:

```
caractere = self.advance()

if caractere in tokens_def.SINGLE_CHAR_TOKENS:
    tipo = tokens_def.SINGLE_CHAR_TOKENS[caractere]
    self.add_token(tipo)
```

Essa tabela fica em `tokens_def.py` e guarda apenas símbolos que sempre têm um único significado:

```
SINGLE_CHAR_TOKENS = {
    "+": PLUS,
    "-": MINUS,
    "*": STAR,
    ";": SEMICOLON,
    "(": LEFT_PAREN,
    ")": RIGHT_PAREN,
    "{": LEFT_BRACE,
    "}": RIGHT_BRACE,
    "<": LESS,
    ">": GREATER,
}
```

Não coloque `=`, `!` e `/` nessa tabela simples. Eles precisam de lógica própria, porque `=` pode virar `=` ou `==`, `!` só é aceito em `!=`, e `/` pode ser divisão ou comentário `//`.

O método `add_token` pega o texto entre `self.start` e `self.current`, cria um `Token` e coloca na lista `self.tokens`. Se sua classe usa `linha` e `coluna`, mantenha esses nomes também na hora de criar o objeto.

Sacada: o scanner não sabe se `x` foi declarado. Ele só sabe que `x` parece um identificador. Declaração é assunto da análise semântica.

Testes do lexer

Antes de fazer parser, teste o lexer isolado.

Entrada:

```
int idade;
idade = 18;
print(idade);
```

Saída esperada:

```
INT_TYPE, IDENT(idade), SEMICOLON,
IDENT(idade), EQUAL, NUMBER(18), SEMICOLON,
PRINT, LEFT_PAREN, IDENT(idade), RIGHT_PAREN, SEMICOLON,
EOF
```


Também teste erro:

```
int x @ 10;
```

O lexer deve acusar caractere inválido @.

Fase 2: análise sintática

O parser recebe tokens e monta uma AST.

O lexer responde:

```
quais símbolos existem?
```

O parser responde:

```
esses símbolos estão em uma ordem válida?
```

Exemplo:

```
x = 10 + 2;
```

Tokens:

```
IDENT, EQUAL, NUMBER, PLUS, NUMBER, SEMICOLON
```

AST:

```
Assign
  name: x
  value:
    Binary +
      Number 10
      Number 2
```

Parser descendente recursivo

Um parser descendente recursivo é feito com funções que lembram a gramática.

Exemplo:

```
def parse_program():
    declarations = []
    statements = []

    while match("INT_TYPE", "BOOL_TYPE"):
        declarations.append(parse_declaration())

    while not check("EOF"):
        statements.append(parse_statement())

    return Program(declarations, statements)
```

Na prática, você pode preferir não consumir o tipo antes de chamar `parse_declaration()`. O importante é entender a ideia: cada função reconhece uma regra.

Outros exemplos:

```
def parse_statement():
    if match("IF"):
        return parse_if()
    if match("WHILE"):
        return parse_while()
    if match("PRINT"):
        return parse_print()
    if match("LEFT_BRACE"):
        return parse_block()
    return parse_assignment()
```

Para expressões:

```
def expression():
    return equality()

def equality():
    expr = comparison()

    while match("EQUAL_EQUAL", "BANG_EQUAL"):
        operator = previous()
        right = comparison()
        expr = Binary(expr, operator, right)

    return expr
```

Essa estrutura cria a AST respeitando a precedência.

Sacada: parser bom não tenta "adivinhar tudo". Ele olha o token atual e decide qual regra aplicar. Se a decisão fica ambígua demais, talvez sua gramática esteja confusa.

AST: a árvore do programa

A AST é uma representação mais limpa do código. Ela remove detalhes que só existiam para o texto ser lido, como parênteses e ponto e vírgula.

Exemplo:

```
print(2 + 3);
```

AST:

```
Print
  Binary +
    Number 2
    Number 3
```

Em Python, você pode usar `dataclasses`:

```
from dataclasses import dataclass

@dataclass
class Program:
    declarations: list
    statements: list

@dataclass
class VarDecl:
    var_type: str
    name: str

@dataclass
class Assign:
    name: str
    value: object

@dataclass
class Binary:
    left: object
    operator: str
    right: object

@dataclass
class Number:
    value: int
```

Você provavelmente terá nós para:

- Program;
- VarDecl;
- Assign;
- If;
- While;
- Block;
- Print;
- Read;
- Binary;
- Number;
- Boolean;
- Variable.

Sacada: a AST deve representar intenção, não pontuação. O nó If precisa saber condição, bloco do then e bloco do else. Ele não precisa guardar os parênteses.

Fase 3: análise semântica

A análise sintática verifica forma. A análise semântica verifica sentido.

Este código pode ser sintaticamente correto:

```
int x;
x = true;
```

Mas semanticamente errado, porque `x` é `int` e `true` é `bool`.

A análise semântica deve verificar:

- variável usada antes de ser declarada;
- variável declarada duas vezes no mesmo escopo;
- atribuição com tipo incompatível;
- condição de `if` e `while` precisa ser `bool`;
- operadores aritméticos exigem `int`;
- operadores de comparação retornam `bool`;
- `print` aceita expressões válidas;
- `read()` retorna `int`, se você definir assim.

Tabela de símbolos

A tabela de símbolos guarda informações sobre nomes.

Exemplo:

```
x -> int
ativo -> bool
```

Uma versão simples:

```
class SymbolTable:
    def __init__(self):
        self.scopes = [{}]

    def define(self, name, var_type):
        current = self.scopes[-1]
        if name in current:
            raise SemanticError(f"variável já declarada: {name}")
        current[name] = var_type

    def lookup(self, name):
        for scope in reversed(self.scopes):
            if name in scope:
                return scope[name]
        raise SemanticError(f"variável não declarada: {name}")
```

Escopos aparecem com blocos:

```
{
    int x;
    x = 10;
}
```

Você pode começar com escopo global apenas. Depois, se quiser melhorar, adicione `begin_scope()` e `end_scope()` para blocos.

```
def begin_scope(self):
    self.scopes.append({})

def end_scope(self):
    self.scopes.pop()
```

Verificação de tipos

A ideia é criar uma função que recebe uma expressão e devolve seu tipo.

```
def type_of_expr(expr):
    if isinstance(expr, Number):
        return "int"

    if isinstance(expr, Boolean):
        return "bool"

    if isinstance(expr, Variable):
        return symbols.lookup(expr.name)

    if isinstance(expr, Binary):
        left = type_of_expr(expr.left)
        right = type_of_expr(expr.right)

        if expr.operator in ["+", "-", "*", "/"]:
            require(left == "int" and right == "int")
            return "int"

        if expr.operator in ["<", ">", "==", "!="]:
            require(left == right)
            return "bool"
```

Para atribuição:

```
var_type = symbols.lookup(assign.name)
value_type = type_of_expr(assign.value)

if var_type != value_type:
    error("tipos incompatíveis")
```

Para if:

```
condition_type = type_of_expr(if_node.condition)
if condition_type != "bool":
    error("condição do if deve ser bool")
```

Sacada: type checking fica muito mais fácil quando a AST está boa. Se sua semântica parece impossível, talvez a AST não esteja representando bem o programa.

Fase 4: código intermediário

O código intermediário é uma forma mais simples e linear do programa.

O enunciado sugere código de três endereços, também chamado de TAC.

Exemplo fonte:

```
x = 2 + 3 * 4;
```

TAC:

```
t1 = 3 * 4
t2 = 2 + t1
x = t2
```

Cada instrução faz pouca coisa.

Você precisa de:

- temporários: t1, t2, t3;
- labels: L1, L2, L3;
- instruções simples.

Exemplo de gerador de temporário:

```
class IGenerator:
    def __init__(self):
        self.temp_count = 0
        self.label_count = 0
        self.instructions = []

    def new_temp(self):
        self.temp_count += 1
        return f"t{self.temp_count}"

    def new_label(self):
        self.label_count += 1
        return f"L{self.label_count}"
```

Para expressão binária:

```
def gen_expr(expr):
    if isinstance(expr, Number):
        return expr.value

    if isinstance(expr, Variable):
        return expr.name

    if isinstance(expr, Binary):
        left = gen_expr(expr.left)
        right = gen_expr(expr.right)
        temp = new_temp()
        emit(f"{temp} = {left} {expr.operator} {right}")
        return temp
```

Para if:

```
t1 = x > 10
ifFalse t1 goto L1
print x
goto L2
L1:
print 0
L2:
```

Para while:

```
L1:
t1 = x > 0
ifFalse t1 goto L2
x = x - 1
goto L1
L2:
```

Sacada: if e while não são mágicos. Eles viram saltos condicionais e labels. Entender isso é uma das partes mais importantes do projeto.

Fase 5: código final com bytecode

Você poderia gerar Assembly x86 ou MIPS, mas para aprender bem e terminar o projeto com qualidade, bytecode próprio é mais direto.

Bytecode é uma lista de instruções para uma máquina virtual que você mesmo cria.

Exemplo fonte:

```
x = 2 + 3;
print(x);
```

Bytecode:

```
PUSH 2
PUSH 3
ADD
STORE x
LOAD x
PRINT
HALT
```

Uma VM baseada em pilha funciona assim:

- PUSH 2: empilha 2;
- PUSH 3: empilha 3;
- ADD: tira dois valores da pilha, soma e empilha o resultado;
- STORE x: tira valor da pilha e guarda em x;
- LOAD x: pega x da memória e empilha;
- PRINT: tira valor da pilha e imprime.

Estado da VM:

```
stack = []
memory = {}
ip = 0
```

ip significa instruction pointer, ou contador de instrução.

Um esqueleto:

```

while ip < len(instructions):
    instr = instructions[ip]

    if instr.op == "PUSH":
        stack.append(instr.arg)

    elif instr.op == "ADD":
        b = stack.pop()
        a = stack.pop()
        stack.append(a + b)

    elif instr.op == "STORE":
        memory[instr.arg] = stack.pop()

    elif instr.op == "LOAD":
        stack.append(memory[instr.arg])

    ip += 1

```

Para desvios:

```

JUMP L1
JUMP_IF_FALSE L2
LABEL L1

```

Na VM, você pode primeiro mapear labels para posições:

```

labels = {
    "L1": 4,
    "L2": 12
}

```

E então:

```

elif instr.op == "JUMP":
    ip = labels[instr.arg]
    continue

elif instr.op == "JUMP_IF_FALSE":
    value = stack.pop()
    if value is False:
        ip = labels[instr.arg]
        continue

```

Sacada: bytecode de pilha é mais simples que TAC para executar. TAC é mais simples para mostrar geração intermediária. Você pode entregar os dois: TAC para o relatório e bytecode para executar.

Como ligar as fases

O arquivo principal do compilador pode seguir este fluxo:


```
source = read_file(path)

tokens = Lexer(source).scan_tokens()
ast = Parser(tokens).parse()

SemanticAnalyzer().analyze(ast)

ir = IRGenerator().generate(ast)
bytecode = BytecodeGenerator().generate(ast)

VM(bytecode).run()
```

Esse fluxo é valioso porque permite testar cada etapa separadamente.

Você pode criar modos de execução:

```
python main.py exemplo.lang --tokens
python main.py exemplo.lang --ast
python main.py exemplo.lang --ir
python main.py exemplo.lang --bytecode
python main.py exemplo.lang --run
```

Isso ajuda muito na apresentação e no relatório.

Estrutura de pastas recomendada

Uma organização simples:

```
compiler/
  token.py
  lexer.py
  ast_nodes.py
  parser.py
  semantic.py
  ir.py
  bytecode.py
  vm.py
  errors.py
  main.py

examples/
  valido_01.lang
  valido_if.lang
  valido_while.lang
  erro_lexico.lang
  erro_sintatico.lang
  erro_semantico_tipo.lang
  erro_semantico_variavel.lang

tests/
  test_lexer.py
  test_parser.py
  test_semantic.py
  test_vm.py
```

Não comece criando todos os arquivos vazios. Crie conforme a fase exigir.

Sacada: testes pequenos substituem adivinhação. Se o lexer funciona sozinho, você não culpa o parser por erro de token. Se o parser funciona sozinho, você não culpa a semântica por erro de AST.

Ordem prática de desenvolvimento

Siga uma ordem incremental:

1. Defina tokens e palavras reservadas.
2. Faça o lexer reconhecer números, identificadores e símbolos simples.
3. Adicione operadores de dois caracteres, como `==` e `!=`.
4. Teste comentários e espaços.
5. Crie classes da AST.
6. Faça parser apenas para declaração e atribuição simples.
7. Adicione expressões com precedência.
8. Adicione `print`.
9. Adicione `if/else`.
10. Adicione `while`.
11. Faça tabela de símbolos global.
12. Adicione verificação de tipos.
13. Gere TAC para expressões e comandos.
14. Gere bytecode.
15. Implemente a VM.
16. Crie exemplos válidos e inválidos.
17. Escreva o relatório explicando cada fase.

Não avance para parser antes de confiar no lexer. Não avance para semântica antes de conseguir imprimir uma AST coerente.

Erros que seu compilador deve reportar bem

Exemplos de erros léxicos:

```
int x @ 10;
```

Mensagem boa:

```
Erro léxico na linha 1, coluna 7: caractere inesperado '@'
```

Exemplo de erro sintático:

```
int x
x = 10;
```

Mensagem boa:

Erro sintático na linha 2: esperado ';' após declaração de variável

Exemplo de erro semântico:

```
int x;  
x = true;
```

Mensagem boa:

Erro semântico: variável 'x' é int, mas recebeu bool

Sacada: mensagem de erro boa vale muito na apresentação. Ela mostra que você não só fez o caminho feliz, mas entendeu o que precisa ser rejeitado.

Mini roteiro mental por fase

Use estas perguntas quando estiver programando.

Lexer:

- Qual caractere estou lendo?
- Ele inicia número, identificador, operador ou pontuação?
- Preciso olhar o próximo caractere?
- Devo ignorar ou gerar token?
- Estou atualizando linha e coluna?

Parser:

- Qual regra da gramática estou tentando reconhecer?
- O token atual é esperado?
- Eu devo consumir esse token agora?
- Qual nó da AST esta regra deve devolver?
- A precedência da expressão está correta?

Semântica:

- A variável existe?
- Ela foi declarada no escopo correto?
- Que tipo essa expressão devolve?
- Esse operador aceita esses tipos?
- Essa atribuição é permitida?

IR:

- Essa expressão precisa de temporário?
- Esse comando precisa de label?

- Onde começa e termina o fluxo de `if`?
- Onde começa e termina o fluxo de `while`?

VM:

- A instrução mexe na pilha, na memória ou no fluxo?
- Ela consome quantos valores?
- Ela produz quantos valores?
- O `ip` deve avançar normalmente ou saltar?

Exemplo completo em fases

Código fonte:

```
int x;  
x = 2 + 3 * 4;  
print(x);
```

Tokens:

```
INT_TYPE IDENT(x) SEMICOLON  
IDENT(x) EQUAL NUMBER(2) PLUS NUMBER(3) STAR NUMBER(4) SEMICOLON  
PRINT LEFT_PAREN IDENT(x) RIGHT_PAREN SEMICOLON  
EOF
```

AST:

```
Program  
  VarDecl int x  
  Assign x  
    Binary +  
      Number 2  
      Binary *  
        Number 3  
        Number 4  
  Print  
    Variable x
```

Tabela de símbolos:

```
x -> int
```

TAC:

```
t1 = 3 * 4  
t2 = 2 + t1  
x = t2  
print x
```

Bytecode:

```
PUSH 2
PUSH 3
PUSH 4
MUL
ADD
STORE x
LOAD x
PRINT
HALT
```

Resultado esperado:

```
14
```

Otimizações simples

Otimização é bônus, mas você pode implementar uma pequena se sobrar tempo.

A mais simples é constant folding, ou simplificação de constantes.

Antes:

```
x = 2 + 3 * 4;
```

Depois:

```
x = 14;
```

Na AST, se um Binary tem dois lados numéricos, você calcula antes:

```
if isinstance(expr.left, Number) and isinstance(expr.right, Number):
    if expr.operator == "+":
        return Number(expr.left.value + expr.right.value)
```

Outra otimização simples é remover código morto:

```
if (false) {
    print(1);
} else {
    print(2);
}
```

Pode virar:

```
print(2);
```

Sacada: só faça otimização depois do compilador funcionar. Otimização em compilador quebrado só torna os erros mais difíceis de enxergar.

O que entregar e como demonstrar

Pelo critério do enunciado, a avaliação se concentra em:

- corretude léxica e sintática;
- análise semântica;
- geração de código;
- documentação e qualidade do código.

Para demonstrar bem:

1. Mostre um programa válido simples.
2. Mostre os tokens gerados.
3. Mostre a AST ou uma impressão textual dela.
4. Mostre a tabela de símbolos ou diga quais símbolos foram registrados.
5. Mostre o TAC.
6. Mostre o bytecode.
7. Execute na VM.
8. Mostre erros léxicos, sintáticos e semânticos.

Exemplos obrigatórios para testar:

```
// válido
int x;
x = 10;
print(x);
```

```
// erro léxico
int x;
x = 10 @ 2;
```

```
// erro sintático
int x
x = 10;
```

```
// erro semântico: variável não declarada
x = 10;
```

```
// erro semântico: tipo incompatível
int x;
x = true;
```

Checklist de independência

Você está entendendo bem quando consegue responder:

- O que o lexer recebe e devolve?
- O que o parser recebe e devolve?

- Por que a AST é diferente da lista de tokens?
- Onde a tabela de símbolos é preenchida?
- Onde o compilador detecta variável não declarada?
- Por que `if (10)` deve dar erro?
- Como `while` vira labels e jumps?
- Como uma expressão vira bytecode de pilha?
- O que a VM guarda na pilha?
- O que a VM guarda na memória?

Se uma resposta estiver confusa, volte para essa fase antes de continuar.

Plano de estudo em 7 dias

Dia 1: gramática, tokens e exemplos da linguagem.

Dia 2: lexer completo e testes manuais.

Dia 3: AST e parser para declaração, atribuição e expressões.

Dia 4: parser para `print`, `if/else`, `while` e blocos.

Dia 5: análise semântica, tabela de símbolos e type checking.

Dia 6: TAC, labels, temporários e bytecode.

Dia 7: VM, testes finais, relatório e exemplos de erro.

Se algum dia atrasar, reduza escopo. É melhor ter `int`, `if`, `while`, `print` funcionando bem do que tentar muitos recursos incompletos.

Conclusão

O compilador que você vai construir não precisa ser enorme. Ele precisa ser bem separado em fases, aceitar programas válidos, rejeitar inválidos e gerar uma forma executável.

O núcleo conceitual é:

```
texto não estruturado
-> tokens
-> árvore
-> árvore validada
-> instruções intermediárias
-> instruções executáveis
```

Se você mantiver essa separação, cada parte fica pequena o suficiente para implementar, testar e explicar.